CO215, Assignment 6

Roll No : CSB23017

Name : Raktim Ranjan Handique

Session : Spring **Year** : 2025

Objective:

1. To learn the basic functions of a debugger-
   a. accessing registers and memory,
   b. editing program and data,
   c. single-stepping, executing program;
2. Observe the following-
   a. the location of the data elements, the location of the code;
   b. the program execution, one instruction at-a-time;
   c. the content of the registers and the flags, the change in them with execution of the instructions;
   d. Observing the execution of branch instructions;
   e. Observing the execution of program loop.

Exercises:

1. **Write an 8086 ALP program to compute the sum of a list of integers given as input by the user .Use conditional and unconditional jump instructions (e.g. jne, jge, jmp, etc) instead loop.**
2. Note the content of all the registers and the flags;
3. Insert data values for the first 5 data elements of the array;
4. Execute the program by single-stepping;
5. **Observe the content of the registers and the flags after every step of execution;**
6. Record the machine code for the program, in hexadecimal, instruction wise;
7. Prepare a report comprising the objectives, exercises carried out, observations, learning outcome of the exercises.

Details of work:

To analyse debugger functions first an 8086 program was written that can compute sum of a list of integers(5 in this case). The loops were replaced with JNE logic i.e. unconditional branching was used. The code is given below:

```
CR EQU 0DH
LF EQU 0AH
MAX_SIZE EQU 5

.MODEL SMALL
.STACK 100H
.DATA
```

```
array DW MAX_SIZE DUP(?)
sum DW 0
prompt_msg DB 'Enter 5 non-zero single digit positive integers.',
CR, LF, CR, LF, '$'
output_msg DB 'Sum of the integers is: $'
ten DB 10

.CODE

main PROC
    .STARTUP
    lea dx, prompt_msg
    mov ah, 09h
    int 21h
    mov bx, OFFSET array
    mov cx, MAX_SIZE
    sub dx, dx

;loop for reading a set of 5 single digit numbers
    rd_loop:
    sub ax, ax
    mov ah, 01h ; to read a character from the keyboard, 1 byte char
gets stored in AL
    int 21h
    sub al, 30h ; to convert from character to number
    mov ah, 0 ; AH = 0, AL = [num] : AX = [num]
    mov [bx], ax
    add bx, 2
    dec cx
    cmp cx,0
    jne rd_loop         ;----- Used unconditional branch instead of
loop

    compute_sum:

    mov bx, OFFSET array
    mov cx, MAX_SIZE
    mov ax, 0

    sum_loop:
    add ax, [bx]
    add bx, 2
    dec cx              ;-----
    cmp cx,0
    jne sum_loop         ;----- Used unconditional branch instead of
loop

    mov sum, ax
; display output message
    lea dx, output_msg
    mov ah, 09h
    int 21h

; print a two digit-number - sum
    sub dx, dx
```

```
    mov ax, sum
    div ten ; al = ax/10, ah = ax%10
    mov bx, ax ; keeping a copy in bx because ax will be used for
display operations
    add bh, 30h ; converting number to character
    add bl, 30h ; converting number to character
    mov dl, bl
    mov ah, 02h
    int 21h
    mov dl, bh
    mov ah, 02h
    int 21h
.EXIT
    mov ah ,4ch
    mov al ,0
    int 21h
    main ENDP

END main
```

Observations:

The following observations were recorded using various tools of the debugger:

Initial Register Contents:

AL	AH	BL	BH	CL	CH	DL	DH	IP	SP	BP	SI	DI	CS	DS	SS	ES
00	00	00	00	00	00	00	00	0000	0100	0000	0000	0000	087C	086C	088A	086C

Variables:

Name	Array	Sum	output_msg	Prompt_msg	ten
Location	0884:0000	0884:000A	0884:0041	0884:000C	0884:005A
Bytes occupied	10	2	25	49	2

Execution:

Step	Instruction Executed		IP	Operand 1		Operand 2		Result		Flags					Remarks
	Assembly	Machine Code		Reg	Value	Reg	Value	Reg	Value	OF	SF	ZF	AF	CF	
1	LEA DX, prompt-msg	BA0C00	0017	DX	0884	-	Prompt msg	DX	000C	0	0	0	0	0	Loads address of pm to DX

2	MOV AH,09H	B409	001A	AH	00	-	09H	AX	0900	0	0	0	0	0	Interrupt 21h, Function 09h – Print a string to the screen
3	INT 21H	CD21	001C	-	21H	-	-			0	0	0	0	0	
4	MOV BX, OFFSET array	BB0000	001E	BX	0000	-	OFFSET array	BX	0000	0	0	0	0	0	Moves array address to BX
5	MOV CX, MAX_SIZE	B90500	0021	CX	0000	-	MAX SIZE	CX	0005	0	0	0	0	0	Moves max size to CX i.e. 5
6	SUB DX,DX	2BD2	0024	DX	000C	DX	000C	DX	0000	0	0	1	0	0	Clears DX
7	SUB AX,AX	2BC0	0026	AX	0900	AX	0900	AX	0000	0	0	1	0	0	Clears AX
8	MOV AH,01H	B401	0028	AH	00	-	01H	AX	0100	0	0	1	0	0	Read a character from standard input
9	INT 21H	CD21	002A	-	21H	-	-	-	-	0	0	1	0	0	
10	SUB AL,30H	2C30	002C	AL	00	-	30H	AX	0131 -> 0101	0	0	0	0	0	Subtract ASCII Value of input
11	MOV AH,0	B400	002E	AH	01	-	00	AX	0001	0	0	0	0	0	Clear upper byte
12	MOV [BX],AX	8907	0030	[BX]	0000	AX	0001	[BX]	0001	0	0	0	0	0	Move first number to first position of Array
13	ADD BX,2	83C302	0032	BX	0000	-	02	BX	0002	0	0	0	0	0	Move BX to point to next array location
14	DEC CX	49	0035	CX	0005	-	-	CX	0004	0	0	0	0	0	Decrement CX after

															every input
15	CMP CX,0	83F900	0036	CX	0004	-	-	-	-	0	0	0	0	0	Compare if 0
16	JNE rd_loop	75EB	0039	-	Rd loop	-	-	BX CX	000A 0000	0	0	0	0	0	Takes all 4 other inputs Jumps back 4 more times
17	MOV BX, OFFSET array	BB0000	003B	BX	000A	-	OFFSE array	BX	0000	0	0	0	0	0	Moves array address to BX
18	MOV CX, MAX_SIZE	B90500	0039	CX	0000	-	MAX SIZE	CX	0005	0	0	0	0	0	Moves max size to CX i.e. 5
19	MOV AX,0	B80000	0041	AX	0001(enterd VALUE)	-	0	AX	0000	0	0	0	0	0	Clears AX
20	ADD AX,[BX]	0307	0044	AX	0000	[BX]	0001	AX	0001	0	0	0	0	0	ADD first element, Later in loop all elements are added
21	ADD BX,2	83C302	0046	BX	0000	-	2	BX	0002	0	0	0	0	0	Move BX to point to next array location
22	DEC CX	49	0049	CX	0005	-	-	CX	0004	0	0	0	0	0	Decrement CX after every input
23	CMP CX,0	83F900	004A	CX	0004	-	-	-	-	0	0	0	0	0	Compare if 0

24	JNE sum_loop	75f5	004D	sum loop	-	-	-	AX BX CX	0005 000A 0000	0	0	1	0	0	Sums all the numbers
25	MOV SUM,AX	A30A00	004F	Sum	0000	AX	0005	Sum	0005	0	0	0	0	0	Mov the value of AX to sum
26	LEA DX, output msg	BA4200	0052	DX	0000	-	Output msg	DX	0042	0	0	0	0	0	Loads output message to DX
27	MOV AH,09H	B409	0055	AH	00	-	09H	AX	0905	0	0	0	0	0	Interrupt 21h, Function 09h – Print a string to the screen
28	INT 21 H	CD21	0057	-	21H	-	-	-	-	0	0	0	0	0	
29	SUB DX,DX	2BD2	0059	DX	0042	DX	0042	DX	0000	0	0	1	0	0	Clears DX
30	MOV AX,SUM	A10A00	005B	AX	0905	-	0005	AX	0005	0	0	1	0	0	Moves sum to AX
31	DIV ten	F6365B 00	005E	-	Ten	-	-	AX	0500	0	0	1	0	0	Divide AX by 10
32	MOV BX,AX	8BD8	0062	BX	000A	AX	0500	BX	0500	0	0	1	0	0	Moves quotient to BX
33	ADD BH,30H	80C730	0064	BH	05	-	30H	BX	3500	0	0	0	0	0	Adds ASCII
34	ADD BL,30H	80C330	0067	Bl	00	-	30H	BX	3530	0	0	0	0	0	ADDS ASCII
35	MOV DL,BL	8AD3	006A	DL	00	BL	30	DX	0030	0	0	0	0	0	Moves first character to DL
36	MOV AH,02H	B402	006C	AH	05	-	02H	AX	0200	0	0	0	0	0	Print a single character from DL to the screen
37	INT 21H	CD21	006E	-	21H	-	-	AX	0230	0	0	0	0	0	

38	MOV DL,BH	8AD7	0070	DL	30	BH	35	DX	0035	0	0	0	0	0	Moves second character to DL
39	MOV AH,02H	B402	0072	AH	02	-	02H	AX	0230	0	0	0	0	0	Print a single character from DL to the screen
40	INT 21H	CD21	0074	-	21H	-	-	AX	0235	0	0	0	0	0	
41	MOV AH,4C	B44C	0076	AH	02	-	4C	AX	4C35	0	0	0	0	0	Terminate the program and return control to the operating system Program ENDS
42	INT 21H	CD21	0078	-	21H	-	-	AX BX CX DX SI BP SP DS ES SS IP	070D 0001 0315 0006 000C 0314 031A 0315 3002 070D 0000	0	0	0	0	0	

Machine code:

Instruction No.	Starting memory location	Machine Code in Hexadecimal
1	0017	BA0C00
2	001A	B409
3	001C	CD21
4	001E	BB0000
5	0021	B90500
6	0024	2BD2
7	0026	2BC0
8	0028	B401
9	002A	CD21
10	002C	2C30
11	002E	B400
12	0030	8907

13	0032	83C302
14	0035	49
15	0036	83F900
16	0039	75EB
17	003B	BB0000
18	0039	B90500
19	0041	B80000
20	0044	0307
21	0046	83C302
22	0049	49
23	004A	83F900
24	004D	75f5
25	004F	A30A00
26	0052	BA4200
27	0055	B409
28	0057	CD21
29	0059	2BD2
30	005B	A10A00
31	005E	F6365B00
32	0062	8BD8
33	0064	80C730
34	0067	80C330
35	006A	8AD3
36	006C	B402
37	006E	CD21
38	0070	8AD7
39	0072	B402
40	0074	CD21
41	0076	B44C
42	0078	CD21

Learning Outcome:

1. Debugger functions and their use:

Answer:

Turbo Debugger (TD) is a tool that comes with TASM and helps us **test, trace, and debug assembly programs**. Since assembly is low-level and a small mistake can crash the program, TD is very useful for students like me who are still learning.

Some functions in Turbo Debugger that were used or tested:

- **Trace (F7)**: This executes the program **one instruction at a time**. It helps me see exactly how each line affects the registers and memory.
- **Step (F8)**: Similar to Step Into, but it **skips over procedure calls**, which is useful when I don't want to go inside a function.
- **Watch Window**: I can add variables or registers here to **see their values update in real-time** as the program runs.
- **Registers Window**: Shows all CPU registers (AX, BX, CX, etc.) so I can track changes after each instruction.
- **Memory Window**: Lets me view and even edit memory. This is useful to check strings, buffers, and data segments.
- **Breakpoints (F2)**: I can set breakpoints to **pause execution at a specific line**, so I don't have to trace everything from the start.

Using Turbo Debugger makes it much easier to **visualize what's happening** inside my assembly programs. It helps me understand how data flows through registers and memory, and it makes debugging much faster and more organized.

2. Location of data and code in a program:

Answer:

In an assembly language program written using TASM, the code and data are stored in different parts of memory, which are called segments. The code segment (defined using `.code`) contains all the instructions that the CPU will execute, such as `mov`, `add`, or `int 21h`. This segment is loaded into the Code Segment Register (CS) during execution. On the other hand, the data segment (defined using `.data`) stores all the variables, constants, and strings used in the program. To access this data correctly, we usually initialize the Data Segment Register (DS) at the beginning of the program using the instructions `mov ax, @data` followed by `mov ds, ax`. This tells the processor where the data is located in memory. Understanding the location of data and code in a program is important because if the segments are not set properly, the program may not run as expected or might access the wrong memory locations.

From the observation we also see that the code(instructions) is stored before the data.

Data starts at the location 0884:000 (start of the ‘array’) variable whereas the code begins from the location 087C:0017(start of first line of code).

3. Observing the machine instructions and their execution:

Answer:

In 8086 assembly, each instruction has a machine code equivalent. For example:

```
MOV AX, 1234H → Machine code: B8 34 12
```

Using Turbo Debugger, execution can be observed step by step. Instructions like:

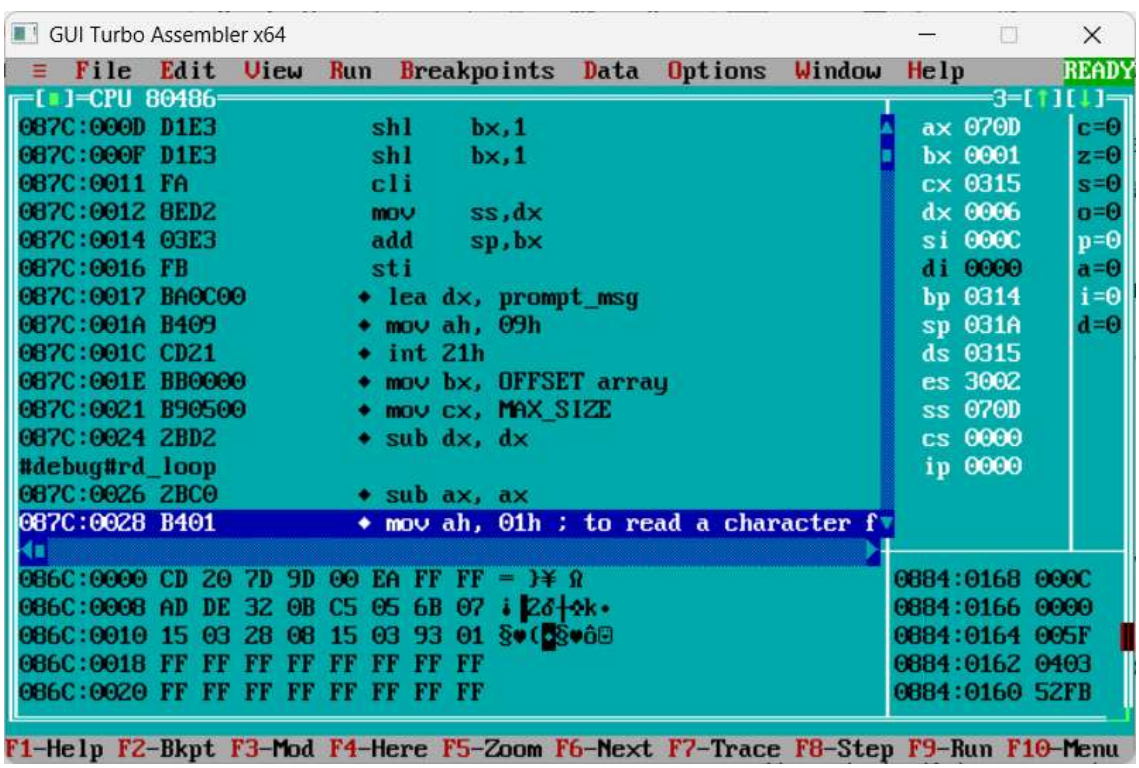
```
ADD AX, BX → Updates AX with the sum of AX and BX
```

Control flow instructions:

```
JNE LABEL → Jumps if the Zero Flag is set
      (visible in the Flags window)
```

DOS interrupt example:

```
MOV AH, 01H                                          .
INT 21H → Waits for key input, stores ASCII code in AL
```

Turbo Debugger shows register and memory changes instantly, helping understand how each instruction affects the system.

Example of how the Debugger screen looks like and its various parts-